

Agile Procurement: New Acquisition Approach to Agile Software Development

Diane Jamieson
Curtin Business School
Curtin University of Technology
Perth, Australia
Diane.Jamieson@Curtin.edu.au

Kevin Vinsen
ADI Limited
Perth, Australia
Kevin.Vinsen@adi-limited.com

Guy Callender
Curtin Business School
Curtin University of Technology
Perth, Australia
Guy.Callender@cbs.curtin.edu.au

Abstract

Contemporary software is increasingly developed using an agile development approach, yet the supplier is generally selected as a result of a waterfall-style competitive tendering and contracting process. The procurement activity may be incompatible with an agile elaboration of requirements and development of functionality, and lead to sub-optimal outcomes.

This paper examines the interaction of the procurement and software development lifecycles, explores potential causes of project or system failures and suggests some improvements based on a successful 10 year project between ADI Limited and the Australian Department of Defence.

1. Introduction

Software can be developed in-house but is more often obtained from vendors either as a package or through bespoke software development services. High value software development services are typically delivered as a result of a competitive tendering and contracting (procurement) process that is intended to select the external supplier offering the best value for money solution to the customer.

The procurement process, in association with traditional organisational budgeting processes, encourages the expectation that a suitable software system will be delivered for the contracted or budgeted price. Theoretically, this requires a procurer to have a good understanding of the scope of a software development project and its likely final cost before releasing a tender to the market, to ensure that the allocated budget will be sufficient to fund the full project. However although the success of IT projects is improving [1], cost overruns still represent major causes of concern.

Information about requirements and their costs is crucial in the early procurement phases. If the software

being sought is extensive or complex a significant level of requirements engineering or business process analysis will be required to support the budgeting process and to provide a basis upon which suppliers can prepare a proposal and price estimate. Subsequent to a contract being signed a further requirements engineering or business process analysis activity would usually be undertaken by the winning tenderer as development of the system is commenced.

It seems however, that requirements gathering methodologies frequently do not acknowledge the impact of the procurement process in separating the activity into two distinct phases: pre-contract award and post-contract award. These phases are probably undertaken by different parties and almost certainly using different approaches. Similarly contemporary agile methods of software development do not appear to consider the role of the procurement process in influencing success, although the agile Unified Process (UP) [2] recommends running projects in two contract phases, each of multiple timeboxed iterations.

UP recommends that phase one is a relatively short fixed-time and fixed-price contract. Its goal is to complete the first few iterations of the project, undertake evolutionary requirements analysis and produce preliminary software. Phase one outputs, including the software, are shared with the tenderers for a phase two fixed-price contract. The evolutionary refinement of specifications and code in phase one provides higher quality data for phase two estimators. However UP tends to be used by suppliers, not by procurers, and large single phase projects are common.

Budget estimation for large software development projects is, in practise, often inaccurate, and the determination of value for money is fraught with problems. In theory, a value for money decision, incorporates factors such as supplier track record, suitability of the technical approach and the whole of life costs associated with that solution. However the concepts of value for money and life cycle costing for software development are described, but poorly

defined, in the practitioner and scholarly literatures. There is extensive anecdotal evidence to suggest that procurers often equate “best value” with lowest price and rarely revisit this decision at the completion of a project. If the winning supplier has misunderstood or misrepresented the costs and the customer is insufficiently skilled to identify the problems and risks, it is quite likely that an alternative supplier would have been preferable.

Prior to signing a contract the winning supplier will have had little opportunity to confirm or fully determine all requirements. There are likely to be hidden requirements, implicit to the procurer but not articulated to the supplier, which will increase the final cost but cannot readily be quantified. In fixed price contracts the primary role of contingency allowances is to anticipate, in an unsophisticated way, the risk of changes occurring in unforeseen circumstances [3]. Agile methods recognise that change is an inevitable outcome of the iterative software development process, as requirements are refined and elaborated and the hidden implicit requirements discovered. Changing requirements must be accepted as normal in software development projects yet will add costs and increase budget pressures.

This paper discusses the interaction between traditional acquisition approaches and agile software development methodologies. An agile procurement approach, which aims to reduce the need for predictive planning and harmonise the acquisition and software development, is proposed.

2. The Procurement Process

2.1. Overview

A competitive tendering and contracting process through which outsourced software development is purchased is typically represented by a 12-step process [4] that covers the phases of contract planning, formation and management. The process is strictly waterfall and each step can only commence when the previous step is completed. During contract planning the organisational need is identified, the broad scope determined and the budget is estimated and sourced. In contract formation a tender is prepared and released by the customer. Suppliers respond and the winning tender is selected subsequent to an evaluation activity. In the contract management phase the software development is delivered under the contract.

A sophisticated procurement organisation will apply a project management methodology across the procurement process, although there is evidence that this approach is not typical in many organisations.

2.2. Contract Planning

During contract planning a business case is produced to justify the project and its budget. Subsequently, a procurement plan will formalise the allocated expenditure, scope, management mechanisms and other necessary details. Typically, the procurement plan creates the rules by which the procurement process and resulting contract are to be managed. The expenditure over the life of the project, identified within an approved procurement plan, represents the budget that has been allocated and the “not-to-exceed” value. Herein lies one key problem encountered in outsourced software development: the budget and broad scope are usually defined long before there can be an adequate understanding of the detailed requirements and therefore the true costs. Jacobson [5] acknowledges that the true cost of software cannot be determined until the design is complete, and although estimation techniques have evolved to create more reliable early estimates, it is apparent from the evidence outlined below, that more information and more sophisticated methodologies are required to both perform elaborate software costing and match procurement processes to the needs of complex costing situations.

For the majority of software development projects it is unlikely the procurer will be able to appropriately understand the requirements before allocating the budget. Other product development domains encourage the practice of adaptive planning [6] over the predictive planning used in software development.

From the software estimation perspective, McConnell [7] suggests that the “cone of uncertainty” is reduced over time when agile development principles are followed. Frequently, as more information becomes available during software development the scope is increased, usually resulting in a need for more budget. Traditionally, many financial policies and procedures require any budget not identified in the procurement plan to initiate an entirely new procurement process. This formal procurement process usually causes significant expense, delays and management problems in a software development project.

2.3. Contract Formation

Ideally, during contract formation, some degree of business process analysis or requirements engineering could be expected to be undertaken by the customer or a specialist contractor, to create the requirements specification documentation to be included with the tender. Tender documentation, typically, contains:

- any technical/technology requirements, such as the customer's IT infrastructure and preferred software technologies;
- details of any complex third party interfaces; and
- pricing requirements, being fixed price, time and materials or a combination of both.

There is anecdotal evidence that greater understanding of the inevitability of change has reduced the prevalence of large fixed price contracts in the public sector. Public sector purchasers have become increasingly aware of the embarrassment caused when large expensive, public funded projects fail or are cancelled and the details are published in the media or released in the wake of a review by an Auditor-General. This evidence suggests that the public sector has embraced the principles in the CHAOS Report [8], favouring small projects that build on one another. Time and materials contracts place a greater onus on the purchaser to achieve organisational outcomes within the allocated budget. They work well for suppliers that produce only what can be paid for, but the purchaser frequently runs out of money before the systems are complete. If an agile development process has been adopted, at the end of the final iteration the purchaser will have software that provides them with some value and achieves some of their functional requirements.

2.4. Post-Award Contract Management

The contract formed at the end of this phase creates the rules upon which the supplier must deliver the system. Frequently in the post-contract contract management phase the supplier will be dealing with the procurer's contract manager who:

- had no role in the preceding procurement activity;
- has little or no knowledge of the requirements of the project; and
- works for a different part of the customer organisation to that which managed the preparation of the request documents.

This contract manager may or may not have any knowledge of requirements engineering and in all likelihood will have no formalised contract management methodology [9]. These deficiencies have the potential to create an adversarial relationship between the buyer and supplier.

3. Evolutionary Acquisition

A system designed to provide a more effective procurement outcome in situations of high product uncertainty was implemented by the US Department of Defense (US DoD) in 2001. This preferred strategy

for the procurement of operational systems was called Evolutionary Acquisition (EA), and is:

an acquisition strategy that defines, develops, produces or acquires, and fields an initial hardware or software increment of operationally useful capability [10].

Note that EA is a strategy (although it is also referred to as a policy [10]) that encourages buying in increments according to affordability, urgency of need and evolving threat, and availability of technology [11]. Spiral development is used in each of the increments of an EA strategy although as a process, EA appears to be incompletely defined. Sylvester & Ferrara [10] define spiral development as "an iterative process for developing a defined set of capabilities". There have been attempts to diagrammatically represent EA using increments and spirals [12], however EA the strategy does not seem to have an associated EA procurement process, although it is likely that it based on the procurement process described above. It is interesting that although the development approach incorporated into EA had been around for some time Sylvester & Ferrara [10] note that it had "not much practical implementation outside of the software development community".

However, so concerned was the US DoD, that even though "there was scant experience with implementing EA for major system development"[10], EA was recommended to improve the situation in defense acquisition where "defense acquisition programs cost too much and take too long to deliver"[10].

EA has also been adopted by the Australian Department of Defence (Australian DoD).

4. Software Cost Estimation

The scope of research into software cost estimation methodologies provides some challenges, as different terminologies are seemingly applied to describe the process, according to the discipline area of the researchers. However, a classification has been developed by Snell [13] which, although lacking any organisational learning classification, does provide a preliminary basis upon which to better understand software cost estimation categories: algorithmic models (the most researched); expert judgement; and machine learning. As machine learning appears to be of limited practical application at this point it has been omitted from this discussion.

4.1. Algorithmic Cost Models

The most pervasive algorithmic cost models that have evolved over the last 20 years measure:

- source lines of code (SLOC), such as COCOMO [14];
- function points [15], such as IFPUG or COSMIC FFP;
- or use case points [16].

The quality of estimates using these methods is heavily dependent upon:

- the experience of the person undertaking the estimation;
- the amount of money and time allocated to the estimation process;
- the extent of requirements engineering undertaken; and
- the availability of historical data on similar projects that allow the model to be calibrated.

There is evidence to suggest that when preparing tenders, customers frequently produce budget projections that reflect the availability of money rather than a predicted cost, and/or have insufficient time or ability to undertake formal algorithmic cost estimation. In addition most tender documentation contains insufficient information to permit suppliers to undertake extensive formal algorithmic methods to compute their tendered prices. Even when both suppliers and the customer do use algorithmic methods they are likely to be applying different methods and will certainly have different skill levels in using these methods.

The SouthernSCOPE methodology [17] is one software development procurement process that includes a cost estimation methodology. However, it appears that this methodology is generally only applied to small or simple development projects.

4.2. Expert Judgement

Some comparatively early research by Heemstra [18] found that 62% of estimators and organisations in a Dutch study used expert judgement, making this technique the most widely applied. The findings of this research has been reinforced by the experience of the authors, having seen this approach applied with varying degrees of success on a number of projects over the last 20 years. Certainly, considering the amount of effort required in using expert opinion compared with algorithmic methods, in the hands of experienced personnel it can be relatively effective and create a positive value for money proposition – that is, produce a result that is good enough to work with for a cost level that is relatively insignificant. However to become experienced, estimators would probably have spent many years examining and evaluating algorithmic methods, would have costed many projects using a number of methods and would

have been able to calibrate their own expert judgement through seeing the success or otherwise of their previous estimations.

An important consideration when evaluating the success of expert judgement is that there should also be an appropriate level of expert judgement applied to all of the associated activities. As a general guide, buyers contracting with suppliers of unique software should:

- anticipate and incorporate change in the contracting process;
 - set a realistic time schedule;
 - manage stakeholder expectations;
 - choose the best supplier using appropriate evaluation methods; and
 - undertake supplier relationship management.
- Conversely, suppliers should:
- use suitable development methodologies;
 - supply suitable personnel; and
 - undertake customer relationship management.

The major challenge is to convert these generic contracting principles into operational tools and activities.

5. Agile Software Development

Contemporary software development techniques and methodologies have become increasingly agile; although agile development is not a new concept. Larman [2] has traced the history of iterative and agile development projects going back to the late 1950's.

Research has shown that the more complex the system the more inevitable is change. Boehm and Papaccio [19] found that a typical software project experienced 25% change in requirements. Jones [20] shows that projects in excess of 10,000 function points could expect as much as 35% change in requirements.

This is because every iteration of a project may uncover hidden requirements and discover new requirements, as a result of process improvement, as the potential of the system is incrementally explored, and as the functionality is better understood.

As discussed in section 2.2, during the first iterations estimates can be highly inaccurate [2] and it is only after the first few iterations have occurred, when additional requirements engineering has been performed, that the true scope of the project can be understood. After these few iterations a customer can also have some meaningful software to examine and explore. This will allow the customer to focus upon further requirements as the capability of the new software system is explored and the requirements clarified.

In 2001 the Agile Alliance was formed to promote 12 principles for agile software development. A number of methodologies, mostly developed before the Agile Alliance was formed, are becoming popular in mainstream software development. Most of these methodologies recommend that the supplier develops the high risk items first to give the customer confidence in the more complex software components.

The agile principles, as defined by the Agile Alliance [21] are:

1. *Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.*
2. *Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.*
3. *Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.*
4. *Business people and developers must work together daily throughout the project.*
5. *Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.*
6. *The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.*
7. *Working software is the primary measure of progress.*
8. *Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.*
9. *Continuous attention to technical excellence and good design enhances agility.*
10. *Simplicity – the art of maximizing the amount of work not done – is essential.*
11. *The best architectures, requirements, and designs emerge from self-organizing teams.*
12. *At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.*

These principles have been reproduced in full as they demonstrate the scope of this approach to software development yet represent key ideas that are not consistently embraced by procurers, viz:

- Anticipate and manage for change (principle 2);
- Continually review business needs (principle 10);
- Have reasonable work expectations of both customer and supplier personnel (principles 5, 8);
- Embrace interim deliveries (principles 1, 3, 7)
- Have suitable communications channels (principles 4, 6); and

- Maintain positive relationships with suppliers (principles 2, 4, 5, 6, 12).

The next challenge is to link these principles to an appropriate procurement methodology.

6. The Harmonisation of Procurement and Agile Methodologies

Agile methods show a high degree of sophistication in accepting the inevitability of change in software projects; and mandating customer collaboration. Agile methods incorporate the use of timeboxed iterative development [2], which involves fixing the iteration end date and not allowing it to change. If, during the course of an iteration, it becomes clear that the scope for the iteration cannot be met within the timeframe, the scope is reduced. The partial growing system always ends in a stable and tested state on the original planned end date.

At the beginning of each iteration the customer and supplier must negotiate what is achievable from the “to do” list given the available resources. Any new requirements discovered during an iteration can be added to the “to do” list and any high value items can be implemented in a subsequent iteration. New requirements are not implemented in the iteration in which they are discovered because this will cause the timebox to fail. Agile methods allow the customer to compute a measure of value for money at the beginning of each iteration, (principles 1, 3 and 7) and to calculate earned value at the end of each iteration. Principle 2 aims to ensure that new requirements do not produce confrontational contract change negotiations. The requirements can be scheduled for subsequent iterations.

However, to be fully effective in an outsourced development, it would be valuable if agile methods could be associated with an agile procurement process, along the lines of the EA philosophy. This would allow budgets to be more accurately estimated incrementally, scope to be incrementally explored, the scale of budget variances to be reduced, and customer-supplier relationships to be improved. Taken to the extreme an agile procurement process could give a customer much greater control over software development, including the potential choice of different suppliers for different components. Furthermore, procurement policies, guidelines and processes, and the capabilities of procurement specialists and decision makers involved in purchasing software development would need radical reappraisal before this would be possible, as an agile procurement process would challenge existing budgetary protocols.

Using an agile timeboxed development approach, the supplier would be able to guarantee the delivery date of each iteration. As the staffing levels would be agreed at the beginning of the iteration when the supplier and procurer agree the functionality to be produced in that iteration, the procurer would have an accurate estimate of the cost of the iteration. After the first few iterations are completed, the supplier and procurer would also be able to estimate more reliably the total number of iterations required to complete the project. As the cost of each of the previous iterations would be known, the procurer and supplier would have a frame of reference for the subsequent estimate of the total cost of the project. Because any new requirements would be prioritised by the procurer and supplier, the procurer would be able to adapt the total budget required by the project. Equally, the procurer could decide not to implement functionality if the incremental cost seemed to be too high for the value that that would be gained within the existing technology platform.

If the selection of requirements to be implemented at the beginning of each iteration were managed using a more sophisticated value for money proposition, any requirements that had not been completed by the end of the project should be in the low priority category. As a study by Johnson [22] has shown that only 45% of features are used, any remaining requirements should be unimportant.

Using a fusion of agile development and agile procurement, the procurer would be able to revisit their original contract planning, budget estimates, contract models, and contract management plans. Should additional funds be required this will be highlighted early in the development process.

Agile principles 2, 4 and 6 aim to ensure that the procurer and the supplier do not have to engage in confrontational negotiations over requirements changes and the scope of the system. The supplier is expected to be pleased to accept changing requirements, the procurer is actively involved in the development of the system and a free and open dialog should exist between the two organisations.

7. Case Study

Since 1994 the Australian DoD has contracted ADI Limited (ADI) to supply software development services for the Joint Command Support System (JCSS), a successful ongoing project that uses an EA approach. Although it does not fully implement adaptive budgeting or agile development it does embrace and demonstrate many principles described in this paper.

7.1. Joint Command Support System

In 1994 the Commonwealth of Australia awarded a two-year contract to ADI to supply a Joint Command Support System. Since then the JCSS has become the core operational management system for the Australian Defence Forces and the project is now in its seventh phase and is valued at over \$US58 million (including a substantial operational infrastructure). Using an EA approach, technology changes and product enhancements have been implemented as part of a through life support and development contract.

The JCSS uses a Wide Area Network and is installed on over 140 servers and 2000 desktops in two dozen locations around Australia. ADI continues to be involved as a major contractor in the development and implementation of JCSS. JCSS provides the desktop tools for Defence Force commanders. These consist of integrated developed and standard desktop (mainly Microsoft) applications. The main developed components are the Common Operating Picture and Defence Messaging applications. The Common Operating Picture is a geographical display of all information of interest to the Defence Force. The picture is synchronised and sanitised so that all commanders are operating from the same information. Drill-down facilities are provided on an object of interest, to give access to all reported and stored information. In support of this picture various tools are provided to allow:

- Overlays - of additional information such as map data, satellite data/images, zones of interest (no-go, firing range, movement corridors, etc), router planning, boundaries of interest, text for pre and post briefings, etc;
- Analysis - of the data presented;
- Filtering - of displayed objects;
- Algorithms - various predictive and navigational algorithms;
- Planning tools;
- Presentation/briefings tools; and
- Message Management - the Defence Messaging application provides tools to assist in formatting standard defence messages (situation reports, intelligence reports, operational orders etc) and for managing reliable distribution and acknowledgment of the messages over the secure defence network.

7.2. Procurement Method

JCSS has an ongoing task list of required features and enhancements. A fixed budget is allocated at the beginning of each year by Australian DoD and the

year is divided into two releases of six months each. At the start of each six month release cycle, representatives of the Australian DoD procurement agency and the operational user community meet to decide what tasks will be undertaken. Each task is a self-contained project that is developed using an iterative development approach which embraces many of the principles listed by the Agile Alliance, although ADI does not claim it to be an agile methodology. The iterations are usually 2-4 weeks in duration, with active involvement from the customer. As this is a military project the degree of ceremony is higher than is usually associated with more “mainstream” agile approaches. Prototypes are delivered to the customer, but a formal delivery of working software is not done until the end of the task.

ADI estimates the work effort when provided with the task list by the Australian DoD. For simple tasks a fixed price estimate is produced, but for high risk or complex tasks, the estimate is performed in two phases similar to the method recommended by UP. During the first phase there is a detailed analysis of the task, development of the requirements and production of basic working software. At the end of this first phase a detailed fixed price estimate, based upon time to complete the task, is given to the Australian DoD, which makes a value for money decision on whether to proceed with the task. All the fixed price estimates are “not to exceed” figures so if the work is completed sooner than estimated the Australian DoD only pays for the time used rather than the estimated time.

Figure 1 shows the percentage over-run or under-run on the last 14 development tasks in 2004 on one element of the JCSS program. As per principles 9, 10 & 11 the JCSS tasking model encourages excellence in the developed software, but as an added advantage passes the financial benefits to the procurer.

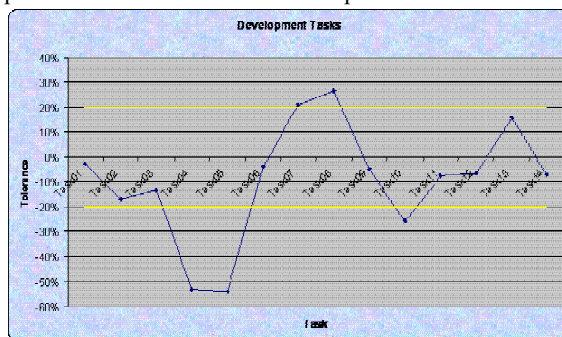


Figure 1: Development Task Performance

This approach has evolved over the last 10 years between the supplier and procurer, to produce a model that contains many of the elements proposed in this

paper. The supplier and procurer have recently celebrated 10 years of partnership, highlighting the success of these arrangements.

JCSS does not demonstrate a truly agile software development however it does follow iterative approaches to both the development and acquisition roles. This represents a well-defined and successful process for the estimation and development of tasks.

8. Conclusion

An agile procurement methodology that is able to be harmonised with agile software development is likely to lead to more successful procurement outcomes for large software systems. This potential has been demonstrated by the success of the JCSS project outlined in this paper. The experience suggests that the application of a traditional waterfall procurement process to an agile software development model is not in the best interests of the procurer or the supplier. Although the Standish Group has reported improvements in software development projects since 1995, budget excesses remain a serious concern and the contribution of the procurement process to these problems appears to be poorly understood. The Standish Group recommended that project size be kept small. Projects that have successfully applied agile methods are now appearing in the literature, and on the Agile Alliance website [23]. These outcomes support the notion of combining an agile procurement model with an agile software development methodology.

Using this method, the procurer would be able, in each iteration, to negotiate the elements that provide the highest value for money at that time. Furthermore, the agile principles aim to ensure that working software that delivers functionality will emerge from every iteration. The application of agile principles ensures that the procurer would have a provisional budget, as well as a loose, global estimate of the total project, that will become more tightly defined as each iteration of the software development is completed. The experiences emerging from the first few iterations will lead to more accurate estimates allowing the procurer the time to revise the procurement planning and secure any additional funding.

Using an agile method and agile procurement, a procurer would be able to add and remove requirements from a “to-do” list, giving them an ongoing ability to redefine requirements. This would encourage the supplier to accept change as normal, thus producing a system that meets the most recently understood needs of the customer rather than those specified in the past.

All through the project the procurer could be fully involved in the specifying, developing and testing of the new system. This would allow the procurer to continually view progress and evaluate the developed functionality for “fitness” for purpose. The outcome is summarised in Table 1 below:

Table 1: Benefits of Agile Procurement

Procurer Benefits	Supplier Benefits
Successful software development procurement outcomes.	Successful delivery of software projects to satisfied customers.
Delivery of software that satisfies the requirements, even if the requirements are dynamic and have changed.	Ability to negotiate requirements and their implementation with the customer.
A managed budget with the ability to forward plan and adapt.	The ability to forward plan development, staffing and cash flow.
Ability to change requirements as the business focus of the organisation changes.	Acceptance of changing requirements without having to manage high risk.

The agile procurement approach introduced in this paper represents a new direction in procurement planning and contract management. Its strength is its capacity to match some major challenges in software development with an agile procurement process that overcomes the traditional waterfall and whole-of-life contracting approach which has, on too many occasions, led to unacceptable cost and time variances in software project outcomes. Such an approach does not provide a comprehensive solution to the problem of software development costing, however it does enable the buyer and the supplier to move towards a collaborative rather than adversarial contracting experience in conditions of high uncertainty.

9. References

- [1] The Standish Group, Latest Standish Group CHAOS Report shows project success rates have improved by 50%. <http://www.standishgroup.com/press/article.php?id=2>. 2003.
- [2] C. Larman, *Agile & iterative development: a manager's guide*. Boston, MA: Pearson Education, Inc, 2004.
- [3] K. Vinsen, D. Jamieson, and G. Callender, "Use case estimation - the devil is in the detail," presented at IEEE Joint International Requirements Engineering Conference RE'04, Kyoto, Japan, 2004.
- [4] State Supply Commission, *The Contract Process*: Government of Western Australia, 2002.
- [5] I. Jacobson, *Object-oriented software engineering - a use case driven approach*. Reading, MA: Addison-Wesley Publishing, 1992.
- [6] J. Highsmith, *Agile project management: creating innovative products*. Boston, MA: Addison Wesley Professional, 2004.
- [7] S. McConnell, *Software project survival guide*. Redmond, WA: Microsoft Press, 1998.
- [8] The Standish Group, Extreme CHAOS. http://www.standishgroup.com/sample_research/PDFpages/extreme_chaos.pdf. 2001.
- [9] D. Jamieson, "Contract management - what is it exactly," *Contract Management in Practice*, vol. 1, pp. 93-97, 2004.
- [10] R. K. Sylvester and J. Ferrara, A., "Conflict and ambiguity: implementing evolutionary acquisition," in *Acquisition Review Quarterly*, vol. Winter, 2003, pp. 3-27.
- [11] Director Operational Test & Evaluation, Evolutionary acquisition: key factors. <http://www.dote.osd.mil/presentations/Coyle20001102/sld007.htm>. 2000.
- [12] K. Farkas and P. Thurston, "Evolutionary acquisition strategies and spiral development processes: delivering affordable, sustainable capability to the warfighters," in *Program Manager*, vol. July-August, 2003, pp. 10-14.
- [13] D. Snell, SCEW. <http://www.ecfc.u-net.com/cost/index.htm>. 1997.
- [14] B. W. Boehm, *Software engineering economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.
- [15] A. J. Albrecht, "Measuring application development productivity," presented at IBM Application Development Symposium, Monterey, CA, 1979.
- [16] G. Karner, "Use case points - resource estimation for objectory projects," Objective Systems SF AB, 1993.
- [17] Multimedia Victoria, SouthernSCOPE. <http://www.mmv.vic.gov.au/southernscope>. 2003.
- [18] F. J. Heemstra, "Software cost estimation," *Information and Software Technology*, vol. 34, pp. 627-639, 1992.
- [19] B. W. Boehm and P. N. Papaccio, "Understanding and controlling software costs," *IEEE Transactions on Software Engineering*, vol. 14, pp. 1462-1477, 1988.
- [20] C. Jones, *Applied software measurement: assuring productivity and quality*, 2nd ed. New York, NY: McGraw-Hill, 1997.
- [21] Agile Alliance, Principles behind the agile manifesto. <http://www.agilemanifesto.org/principles.html>. 2001.
- [22] J. Johnson, "Keynote Speech," presented at XP 2002, Sardinia, Italy, 2002.
- [23] Agile Alliance, Case Study. <http://www.agilealliance.org/articles/searchResults?topic=Case+Study>. 2004.